

Imitation Learning for Mobile Robots: Vision-only Particle Filter and Pursuit Control

Tammie Nwaochei, Vishnu Sankarakumar, Tadeas Horn

Abstract—This report documents our autumn term project for the module Intelligent Robotics. During this project, we implemented a vision-only imitation system in a Webots simulation. The work was further divided into three parts: marker-based perception, particle-filter estimator, and pure-pursuit tracking. Throughout this report, we describe the algorithms used, the experimental design in three worlds of our own design, the representative results, and the limitations of our project.

I. INTRODUCTION

We chose imitation from visual observation because this method is attractive when instrumentation is unavailable, but demonstrations exist. Our student robot follows the teacher in a cluttered arena; however, this could be further explored in larger spaces or with more complex robots. In our example of imitation from visual observation, a teacher robot drives a path while a student observes its movements via a forward-facing camera. The teacher is equipped with a marker, and its detections produce noisy bearing and scale proxies. Using these along with the odometry feed and particle filter, we should be able to estimate the teacher’s pose for a pure-pursuit controller. We utilise two ‘Pioneer 3-DX’ robots in a standard-sized arena in Webots. They provide a good starting point for their size and flexibility.

II. RELATED WORK

A necessary component of our algorithm is a particle filter. These are the standard probabilistic approaches to non-linear state estimation and localisation, widely used in the robotics industry. A particle filter becomes particularly useful for pose tracking under noisy sensor measurements [3], in our case, a low-quality camera image. We also relied heavily on pure-pursuit control, which is a typical feedback steering law that enables vehicles to track a reference path. It can do so by aiming at a look-ahead point [17], and adjusting its path accordingly.

From our research, visual imitation learning has been explored via behaviour cloning, such as supervised learning of observation-to-action mappings. Another option achieving a similar result is visual servo-ing, revolving around direct visual feedback control. As a team, we have decided to bridge these by using visual perception to infer the teacher’s state through filtering and later applying a pure-pursuit tracking algorithm.

III. ALGORITHMS AND METHODS

A. Perception: Marker Detection

The teacher robot is marked with a magenta square patch on its back surface. The student is observing the marker

via a monocular camera placed in front. A hue-based HSV detector configurable bounds per experiment identifies the marker in each frame, producing blob centroid (u, v) in image coordinates and area. From this, we extract the bearing angle relative to the camera optical axis. We also extract a proxy range via the blob area with Gaussian noise: $\sigma_{\text{bearing}} = 0.1$ rad, $\sigma_{\text{range}} = 0.1$ m.

Algorithm 1 Marker detection (per frame)

Require: RGB image I , HSV bounds (h_ℓ, h_u) , saturation threshold $s_{\min}$, value thresholds $(v_{\min}, v_{\max})$, area gate $A_{\min}$

- 1: Convert to HSV: $(H, S, V) \leftarrow \text{HSV}(I)$
- 2: Threshold magenta: $M \leftarrow \mathbb{I}\{h_\ell \leq H \leq h_u, S \geq s_{\min}, v_{\min} \leq V \leq v_{\max}\}$
- 3: De-noise mask: $M \leftarrow \text{close}(\text{open}(M))$
- 4: Largest blob statistics $\Rightarrow$ centroid (u, v) and area A
- 5: **return** $(u, v, A, \text{visible} = \mathbb{I}[A \geq A_{\min}])$

1) *Hypotheses:* **H1:** A vision-only particle filter enables accurate imitation without external localisation, lowering DTW and lateral error.

H2: Increasing perception fidelity (frame rate) improves accuracy and smoothness.

H3: Look-ahead pure-pursuit yields smoother and more stable imitation than a reactive bearing-only baseline.

2) *Measurement model (pixels $\rightarrow$ bearing/range):* Let (c_x, c_y) be the principal point and (f_x, f_y) the focal lengths. Bearing (left positive) follows the pinhole model

$$\theta = \arctan\left(\frac{u - c_x}{f_x}\right), \quad \sigma_\theta^2 = \left(\frac{\sigma_{\text{pix}}}{f_x}\right)^2, \quad (1)$$

where σ_{pix} is the pixel noise scale. A gated proxy range uses apparent size. With $\hat{s} = \sqrt{A}$ and marker side L ,

$$\hat{r} = \frac{f_x L}{\hat{s}}, \quad \sigma_r^2 \approx \left(\frac{\sigma_{\text{pix}} f_x L}{\hat{s}^2}\right)^2, \quad (2)$$

used only when $A \geq A_{\min}$ (otherwise $\hat{r}$ is omitted). The output is $\text{Perceptions}(t, \theta, \hat{r}, \sigma_\theta^2, \sigma_r^2, \text{visible})$.

3) *Occlusion-aware tracking:* An exponential moving average (EMA) stabilises $(\theta, \hat{r})$ and a miss counter provides hysteresis: after N_{lose} consecutive frames with $\text{visible} = 0$ the track is marked lost; on reacquisition the first update uses higher smoothing (lower α) to prevent snapping to noise/false positives.

4) *Public API and ablations:* Configuration exposes two ablations for H2: (i) observation rate $f \in \{5, 10, 20\}$ Hz via processing throttling, and (ii) feature level $L \in \{1, 2\}$, where $L=1$ provides bearing-only and $L=2$ enables the range proxy when the area gate is satisfied.

B. State Estimation: Particle Filter

To estimate the teacher's relative pose $\mathbf{x} = [x, y, \theta]^T$, we use a particle filter with $N = 200$ particles. At each time step, the prediction step propagates particles using a kinematic motion model driven by the student's odometry $(\Delta d, \Delta \theta)$. To account for control noise, Gaussian noise is injected into the control inputs with standard deviations $\sigma_d = 0.05$ m and $\sigma_\theta = 0.05$ rad. This ensures the filter maintains a diverse set of hypothetical locations (particles) around the true pose.

At every single time step, each particle is first moved forward using the teacher's odometry measurements $(\Delta d, \Delta \theta)$. If the teacher's motion estimate is a little bit off, this noise keeps the filter flexible and prevents all the particles from collapsing into the incorrect location.

After the prediction step (when marker is detected), each particle's weight is adjusted according to a Gaussian likelihood function that compares a particle's predicted bearing/range with visual observation. Particles that align with the camera measurement get higher weights, and others are conversely suppressed.

Once all the weights are normalised, we calculate the effective sample size

$$N_{\text{eff}} = \frac{1}{\sum_i w_i^2},$$

which indicates how many of the particles are still realistic. If N_{eff} drops below $N/2$, multinomial resampling is used to eliminate low-weight particles and duplicate the high-probability ones.

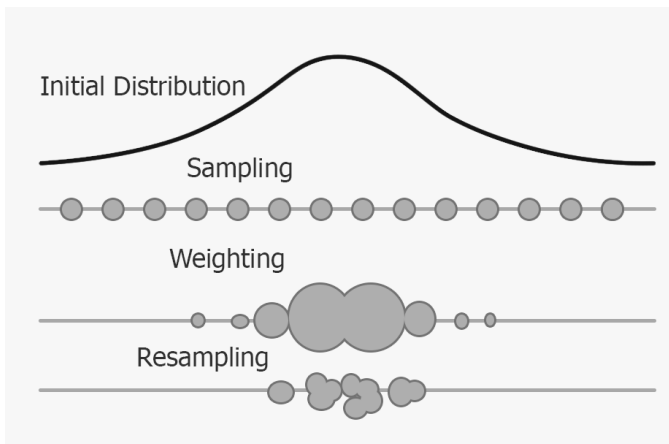


Fig. 1. Visual representation of the sampling process.

Lastly, the weighted average of all particles is used to calculate the overall estimated position and heading,

$$\hat{\mathbf{x}} = \sum_{i=1}^N w_i \mathbf{x}_i.$$

Algorithm 2 Particle Filter (per time step)

Require: Particles $\{x_i, w_i\}$, odometry $(\Delta d, \Delta \theta)$, observation z

- 1: **for** $i = 1 \dots N$ **do**
- 2: $x_i \leftarrow \text{propagate}(x_i, \Delta d, \Delta \theta) + \mathcal{N}(0, Q)$
- 3: $w_i \leftarrow p(z | x_i)$
- 4: **end for**
- 5: Normalize weights: $w_i \leftarrow w_i / \sum_j w_j$
- 6: **if** $1 / \sum_i w_i^2 < N/2$ **then**
- 7: resample()
- 8: **end if**
- 9: **return** $\hat{x} = \sum_i w_i x_i$

This smooth output is supplied directly to the pure pursuit controller so that the student can follow the teacher's heading.

C. Control: Pure-Pursuit and PID

Given the estimated pose $\hat{x}$ and horizon-based predicted path, pure-pursuit computes angular velocity to steer toward a lookahead distance $L_d = 0.8$ m at a following distance of $d_{\text{targ}} = 0.5$ m. Its linear velocity is regulated by PID feedback on distance error: $K_p = 0.5$, $K_i = 0.01$, $K_d = 0.1$, max speed $v_{\text{max}} = 1.2$ m/s.

Algorithm 3 Control: Pure-Pursuit + PID

Require: Pose $\hat{x}$, path, L_d , d_{targ} , gains K_p, K_i, K_d , v_{max} , Δt

- 1: Find lookahead point $\mathbf{p}_{LA}$ on path at distance L_d
- 2: Heading error: $\alpha \leftarrow \text{atan2}(p_{LA,y} - \hat{y}, p_{LA,x} - \hat{x}) - \hat{\theta}$
- 3: Angular velocity: $\omega \leftarrow \frac{2v \sin \alpha}{L_d}$
- 4: Estimate distance: $d \leftarrow \|\hat{\mathbf{p}}\|$ (where $\hat{\mathbf{p}}$ is position vector of $\hat{x}$)
- 5: Distance error: $e \leftarrow d - d_{\text{targ}}$
- 6: PID update: $I \leftarrow I + e\Delta t$, $D \leftarrow (e - e_{\text{prev}})/\Delta t$
- 7: Linear velocity: $v \leftarrow K_p e + K_i I + K_d D$
- 8: Clip: $v \leftarrow \max(-v_{\text{max}}, \min(v, v_{\text{max}}))$, $e_{\text{prev}} \leftarrow e$
- 9: **return** (v, ω)

1) *PID Controller:* A Proportional-Integral-Derivative (PID) controller is used to maintain the target following distance. This controller continuously adjusts the robot's linear velocity (v) based on the instantaneous distance error $e(t) = d(t) - d_{\text{targ}}$.

- **Proportional** (K_p) drives the robot faster when the gap is large and reverses when the gap is too small.
- **Integral** (K_i) ensures the robot does not settle at a distance slightly larger or smaller than the target.
- **Derivative** (K_d) Responds to the rate of change of the error, reducing overshoot and preventing oscillation as the robot converges on the target distance.

2) *Integral Anti-Wind-up:* To prevent the 'Integral Wind-up' phenomenon, where the integral term accumulates substantial error when acceleration is saturated beyond maximum velocity, a clamping-based anti-wind-up mechanism was implemented. If the computed control output is larger

than the max limit of the motor ($v_{max} = 1.2m/s$), integrator updates are paused or reversed. This mitigates exploding error accumulation and maintains controller responsiveness.

3) *Pure Pursuit Steering*: The Pure Pursuit algorithm enables lateral control. The algorithm calculates the curvature (κ) required to move the robot from its current position to a specific "lookahead point" on the teacher's path.

- **Lookahead Search** The controller searches the predicted path horizon for the point closest to a fixed lookahead distance ($L_d = 0.8$ m) from the robot's centre.
- **Curvature Calculation** By defining a circular arc that connects the robot's current point to the lookahead point, the required curvature is calculated using the lateral distance from the target point in the robot's coordinate frame.
- **Send Command** The final angular velocity command (ω) is computed by scaling the curvature by the robot's current linear velocity (v)

4) *Collision Avoidance*: The PID Controller has a *TARGETDISTANCE* parameter currently set to 0.5m. Suppose the robot gets too close ($< 0.5m$). In that case, the error becomes negative, causing the PID controller to naturally output a negative velocity to reverse/slow down, thereby increasing the following distance back to the threshold.

IV. EXPERIMENTAL SETUP

We again used the Webots software to design three obstacle courses of increasing difficulty. To make the results more measurable, we kept the arenas of the same size.

- **Easy**: No obstacles at all. Empty arena
- **Moderate**: Baseline obstacles similar to a default arena introduced earlier in class. Box placement results in occasional occlusion of the teacher marker.
- **Maze**: Highest difficulty tested so far, arena is a maze with tight corners, resulting in easier marker loss along with less predictable teacher movements.

A. Platform and Implementation Details

As defined by the coursework assignment, this project was implemented on Webots R2025, and we got a chance to test it on both Windows and macOS. The simulation uses the standard 'Pioneer 3-DX' model with differential drive kinematics. Both teacher is equipped with a large magenta marker, while the student robot is equipped with a front-facing monocular camera. The teacher navigates the arena with a basic collision avoidance algorithm, while the student is autonomous and relies heavily on the visibility of the marker.

V. EXPERIMENTAL EVALUATION

For all three courses of various difficulties of navigation, we conducted 10 independent trials with the student robot initialised behind the teacher at a fixed offset. The perception system runs at frame rates $\{10, 25, 50\}$ Hz to probe performance across computational budgets. We measure:

- **Lateral deviation**: mean Euclidean distance between the student's position and the nearest point on the teacher's ground-truth trajectory.
- **Heading error**: mean absolute difference between the student's orientation and the teacher's orientation.
- **DTW distance**: similarity measure that quantifies the alignment between the student's path shape and the teacher's path.
- **Control effort**: total energy expenditure approximated by the sum of squared linear and angular velocities ($\sum v^2 + \omega^2$) - a proxy for smoothness and mechanical efficiency.

VI. RESULTS

Figures along with detailed quantitative results such as mean $\pm$ std over 10 trials per condition are based on our experimental runs. Here are several key qualitative observations:

- **Sensitivity to Observation Rate and Effects of Noise**: High-frequency perception (50 Hz) is evidently critical if maximum accuracy in complex environments is the main goal, yielding the smallest lateral deviation of 0.174 m on the Maze map. Reducing the rate to 10

TABLE I
CONSOLIDATED EXPERIMENTAL RESULTS ($\mu \pm \sigma$)

Map Type	Metric	10 Hz	25 Hz	50 Hz
Easy	Lateral Dev. (m)	0.224 ± 0.030	0.212 ± 0.028	0.321 ± 0.031
	Heading Error (rad)	0.568 ± 0.044	0.616 ± 0.013	0.515 ± 0.009
	DTW Distance	0.512 ± 0.057	0.440 ± 0.070	0.241 ± 0.034
	Control Effort	244.7 ± 15.0	316.3 ± 110.3	268.7 ± 5.4
Moderate	Lateral Dev. (m)	0.260 ± 0.054	0.211 ± 0.018	0.318 ± 0.117
	Heading Error (rad)	0.820 ± 0.196	0.691 ± 0.008	0.702 ± 0.160
	DTW Distance	0.591 ± 0.368	0.269 ± 0.015	0.381 ± 0.386
	Control Effort	688.6 ± 36.1	269.2 ± 3.9	324.3 ± 36.1
Maze	Lateral Dev. (m)	0.430 ± 0.019	0.213 ± 0.016	0.174 ± 0.014
	Heading Error (rad)	0.481 ± 0.004	0.609 ± 0.033	0.570 ± 0.015
	DTW Distance	0.105 ± 0.017	0.156 ± 0.020	0.199 ± 0.019
	Control Effort	536.9 ± 15.5	516.4 ± 37.5	450.0 ± 48.6

Hz caused accuracy to degrade significantly in the Maze (0.430 m), confirming that low-latency feedback is paramount for managing frequent turns and occlusions.

- **Control Effort Efficiency:** The most efficient operation is on the Easy map at 10 Hz, requiring a small control effort of 244.7. This demonstrates that for simple trajectories lacking complex obstacles, the system can aggressively cut down computational cost without compromising job completion. As such, a dynamic observation rate could be a good future optimisation.
- **Why Not Always 10 Hz?** While lower perception frame rates reduce computational load, there is a substantial performance trade-off. The 10Hz condition was very unstable on the Moderate map. It required a massive control effort of 688.6 ± 36.1 , which is more than double the 25Hz or 50Hz conditions. This shows the 10Hz update rate introduces significant latency, resulting in detecting turns too late, going wide, and doing aggressive corrections. So, while 10Hz is sufficient for the Easy map (lowest control effort of 244.7), it is much too unstable for anything more complex.
- **Optimal Middle-ground:** The 25 Hz rate yielded the most balanced performance across all metrics. On the Moderate map, it achieved a low lateral deviation of 0.211 m with very good stability (± 0.018 m). 25 Hz is the 'sweet spot' for general use, avoiding the cost of 50 Hz while preventing the instability/failure at 10 Hz. Another piece of evidence supporting this: heading error

has exceptionally low variance across all maps at 25 Hz (e.g. $\sigma = \pm 0.008$), meaning the 25 Hz robot is turning smoothly and confidently, effectively filtering noise without sacrificing the reactivity needed for complex paths.

However, an anomaly was observed in Control Effort. 25Hz suffered from massive variance on the Easy map ($\sigma = \pm 110.3$). This contrasts with the 50 Hz condition, which remained consistent on the Easy map ($\sigma = \pm 5.4$). This suggests that while 25 Hz is a valid cost-saving compromise, it lacks the consistency of the higher 50 Hz rate.

- **High Accuracy or Corner-Cutting?** Surprisingly, the 10 Hz condition got the best DTW score on the Maze map (0.105) despite having the worst lateral deviation (0.430 m). This implies that the laggy 10 Hz robot systematically cut corners through the maze by understeering. This produced a smooth path that matched the teacher's path closely (low DTW) but failed to maintain the right following distance (high lateral deviation). On the other hand, the 50 Hz robot tracked the teacher very closely (low deviation), but micro-corrections likely made the DTW metric much bigger. Consequently, the lower DTW metric for the 10Hz condition should not be mistaken for better performance. It has simply traded path accuracy for path smoothness

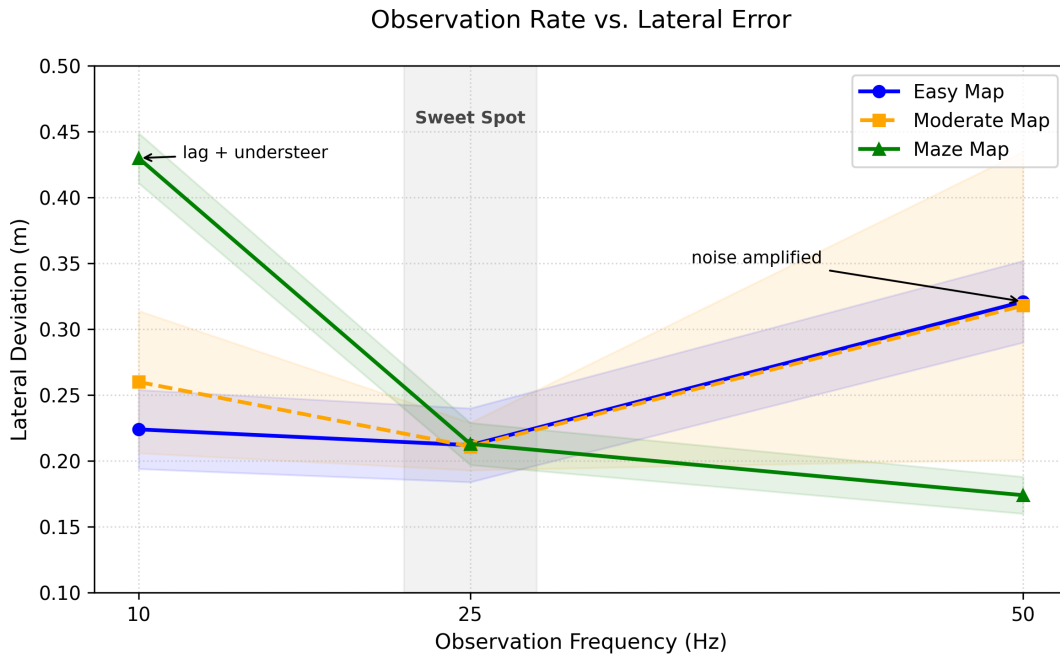


Fig. 2. **Effect of Observation Rate on Tracking Accuracy:** Graph heavily supports the discussed trade-off between latency and noise. High-frequency (50Hz) mitigates lag and understeering in the Maze (green) but suffers in accuracy on the Easy map (blue) due to high noise amplification. The converse is true for 10 Hz. The 25Hz condition (shaded region) identifies the 'sweet spot', maintaining high reactivity without overwhelming noise.

VII. LIMITATIONS

- **Recovery behaviour:** Long occlusions (> 5 s typical) cause recovery behaviour to start, in which the student sweeps the environment by turning on the spot until the teacher is reacquired. This is largely a brittle activity, as a long occlusion usually means the teacher has turned a corner. Pivoting on the spot is too crude a design for consistent, effective recovery.
- **Linear Motion Extrapolation:** The particle filter's trajectory prediction assumes the teacher maintains a constant heading, projecting a straight line from the current estimated pose and ignoring any potential angular velocity completely. Consequently, during sharp turns (as seen in the Maze map), the system predicts a straight path rather than a curve, causing the Pure Pursuit controller to aim for a lookahead point that cuts the corner, resulting in systematic corner-cutting.
- **Single-marker design:** Relies entirely on one coloured patch attached to the teacher. Ambiguous scale (blob area does not uniquely determine distance). No loop closure or global localisation; only relative pursuit.
- **Teacher path assumption:** No explicit interaction or mutual awareness between teacher and student. Assumes teacher moves at moderate speed; rapid acceleration or sharp manoeuvres may exceed the student's tracking bandwidth.
- **Static colour thresholding:** The current marker system uses fixed integers for HSV thresholds. This does not generalise well in the real world.

VIII. FUTURE WORK

- **Robust feature tracking:** Replace single marker with SIFT/BRIEF descriptor-based tracking for invariance to scale, occlusion, and rotation.
- **Learned motion models:** Train an RNN or temporal convolutional network to predict teacher motion (trajectory) during occlusion, reducing divergence risk.
- **Dynamic observation rate:** Ability to automatically change observation rate to fit the given scenario and maximise performance.
- **Intelligent Recovery:** Ability to leverage particle filter estimator's memory to intelligently reacquire the teacher, rather than pivoting on the spot or resorting to naive random walks.

IX. CONCLUSIONS

We have managed to implement a complete, modular vision-based imitation pipeline for mobile robot pursuit. By combining marker-based perception, particle filtering for state estimation, and pure-pursuit feedback control, we enabled a student Pioneer 3-DX to follow a teacher using only onboard monocular vision.

Additionally, while lower perception rates reduce computational load, they introduce a substantial performance penalty

in dynamic scenarios, introducing significant latency, causing the robot to detect turns too late, drift wide, and execute aggressive, energy-intensive corrections to recover.

The experimental results provide strong validation for H1, demonstrating that a vision-only particle filter can achieve high-precision imitation (with lateral deviations as low as 0.174 m) in complex environments without reliance on external localisation.

H2 (increasing perception fidelity improves accuracy and smoothness) was supported in certain cases, though with key nuances. The data confirmed that higher observation rates (50 Hz) are critical for maximising accuracy and minimising control instability; the 10 Hz condition struggled significantly on the Moderate map, requiring more than double the control effort of higher rates.

However, the relationship between fidelity and "smoothness" (DTW) is nuanced. While higher rates provided consistent control, the lag at 10 Hz caused the robot to cheat systematically, artificially improving its DTW score even though accuracy was poor.

Most importantly, the experimental ablation of observation rates ultimately revealed a fundamental trade-off between latency minimisation and noise amplification. While high-frequency perception (50 Hz) proved critical for minimising lateral error in complex environments, the results identified 25 Hz as an excellent balance, maintaining low lateral deviation and stability without the computational cost or noise sensitivity associated with maximum rates.

Design implications.

- Use **50 Hz** in dense, turn-rich layouts where quick reactivity outweighs noise amplification.
- Default to **25 Hz** for balanced accuracy, stability, and effort across heterogeneous maps.
- Reserve **10 Hz** for simple, open layouts or low-power regimes, with conservative speeds and tighter recovery logic.

Experimental results quantify performance across difficulty levels and frame rates, revealing both the effectiveness of the approach in structured environments and its limitations under occlusion and clutter. This project provides a concrete platform for exploring visual imitation, collision avoidance, and multi-robot coordination. Future extensions toward descriptor-based tracking and learned motion priors hold promise for robustness in real-world deployment.

REPRODUCIBILITY

Code and worlds are available in the repository:
<https://github.com/tadun/imitation-learning>
Submission is found in the 'final-submit' branch only.

ACKNOWLEDGMENTS

We would like to thank the Intelligent Robotics course staff at the University of Birmingham and the Webots maintainers.

REFERENCES

- [1] Bradski, G. "The OpenCV Library." *Dr. Dobb's Journal of Software Tools*, 2000.
- [2] Dimitrov, Martin. "Particle Filters (Part 1)." <https://www.lancaster.ac.uk/stor-i-student-sites/martin-dimitrov/wp-content/uploads/sites/31/2021/05/sampling.png>.
- [3] Djuric, Petar M., et al. "Particle Filtering." *IEEE Signal Processing Magazine*, vol. 20, no. 5, 2003, pp. 19–38.
- [4] Dudek, Gregory, and Michael Jenkin. "Differential Drive Kinematics." *CS W4733 NOTES*, 2010.
- [5] Gardner, E. S. "Exponential smoothing: The state of the art—Part II." *International Journal of Forecasting*, vol. 22, no. 4, pp. 637–666, 2006.
- [6] Garrido-Jurado, R., R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez. "Automatic generation and detection of highly reliable fiducial markers under occlusion." *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [7] Gonzalez, R. C., and R. E. Woods. *Digital Image Processing*, 4th ed. Pearson, 2018.
- [8] Hartley, R., and A. Zisserman. *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge Univ. Press, 2004.
- [9] Jain, Harshit, and Priyal Babel. "A Comprehensive Survey of PID and Pure Pursuit Control Algorithms for Autonomous Vehicle Navigation." *arXiv*, 2024, arXiv:2409.09848.
- [10] Köhler, Johannes, Alain Pagani, and Didier Stricker. "Detection and Identification Techniques for Markers Used in Computer Vision." *Visualization of Large and Unstructured Data Sets*, Schloss Dagstuhl, 2011.
- [11] Macenski, Steve, Shrijit Singh, Francisco Martin, and Jonatan Gines. "Regulated Pure Pursuit for Robot Path Tracking." 2023.
- [12] Martínez, Salvador Ibarra, et al. "An Autonomous Navigation Methodology for a Pioneer 3DX Robot." *Computer Technology and Application*, vol. 5, no. 2, 2014.
- [13] Martins, Felipe N. "Set of tutorials to practice robotics concepts with Webots and Python." *Go-to-goal behavior with PID*, 2025.
- [14] MATLAB Documentation. "Anti-Windup Control Using PID Controller Block." <https://www.mathworks.com/help/simulink/slref/anti-windup-control-using-a-pid-controller.html>.
- [15] Michel, Olivier. "Cyberbotics Ltd. Webots™: Professional Mobile Robot Simulation." *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, 2004, p. 5.
- [16] Soille, P. *Morphological Image Analysis: Principles and Applications*, 2nd ed. Springer, 2003.
- [17] Thrun, Sebastian, and John J. Leonard. "Pure Pursuit and Path Tracking." *Mobile Robot Control*, 2013.
- [18] Thrun, Sebastian, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.